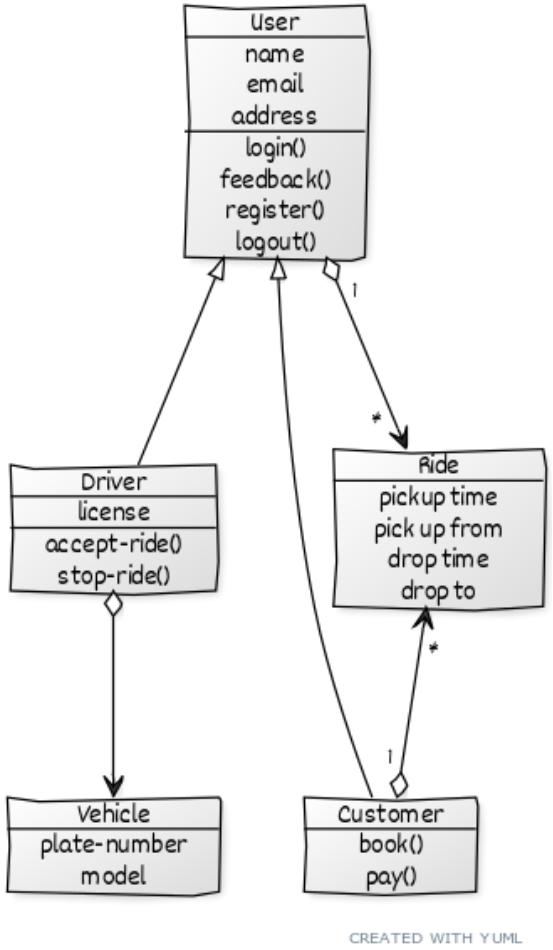
Design & Implementation

Domain Model

- Creating a domain model is paramount to establish a common language
- Try to keep it normalized at first
- Can be Class Diagrams or ER models
- The actual implementation might vary (e.g., CQRS)



Domain Model

Design a domain model for your project

API, Contracts, and interfaces

- An (A)pplication (P)rogramming (I)nterface defines a contract for a client. An implementation with comply with this interface but the client does not see it.
- Define contracts/API early in the implementation
- Do it first at the service level and progressively refine
- It helps // the work and set the boundaries of the components
- Accept that it will change significantly at the beginning. Your understanding of the problem is evolving, so is your API.
- Use a tool to document and maintain your APIs (e.g., Javadoc, Swagger)
- Think as you would want to use it (easy, hard to misuse, comprehensive)
- Do not abuse method chaining

API, Contracts, and interfaces (cont'd)

- Minimize package coupling
- Use interfaces over abstract classes
- Avoid statics, Beware of Singleton (more on that later)
- Stay away from implementation details in the API
- Use standard runtime Exception (IllegalArgument, IllegalState,….)
- Use consistent wording and form
- Use varargs, enums, default
- THINK TESTABILITY

http://www.cs.bc.edu/~muller/teaching/cs102/s06/lib/pdf/api-design?spm=ata.13261165.0.0.38067cd1w5lGp5

Java APIs

Design a Java API for the 2 main use cases of your project

REST APIs

- A REST API is an API that follows the principles of (RE)presentational (S)tate (T)ransfer.
- It is a set of constraints that web services must conform to. They enable scalability, portability, Reliability, Modifiability.

1) Client-Server : separate the UI from the command. Session state is kept on client side. Assume a disconnected system
2) Statelessness : each request from the client to the server must be self-contained
3) Cacheable : if a response cannot be cached (idempotency), it must be explicit
4) Uniform Interface : Identification of resources, manipulating resources through representation, self-descriptive message and hyperlinks as the engine
5) Layered System : see architecture patterns
6) Code on demand (optional) : server can return logic (such as JavaScript)

REST APIs

- Resource : information that can be name with a resource identifier. Use plural nouns not verbs

/employee/123 : employee with id 123

- Representation of the resource : A representation consists in data, metadata and hyperlinks that enables the client to navigate between the states. Usually JSON or XML
- Resource methods: operations that transition the state. Usually it relies on the HTTP methods (GET, PUT, POST, DELETE)

POST employee/123 (with body in either json or xml) creates employee 123

GET employee returns all employees

GET employee/123 returns employee with id 123

PUT employee/123 (with body in either json or xml), updates employee 123

DELETE employee/123 delete the employee with id 123

Guidelines

- User plural nouns, not verbs : /employees and not /getEmployees
- Use parameters for configuring the service BUT NOT to identify resources /employees/1 and not /employees?id=1 BUT /employees?name=XX and not /getEmployeeByName/XXXX
- Use the right HTTP return code : for instance 201 as a return code for a newly created resources and not 200
- Use versioning /v1/employees, /v2/employees
- Use pagination as parameters : /employees?limit=25&offset=50
- Use the OpenAPI specification : https://swagger.io/resources/open-api/

REST APIs

Design a REST API for the 2 main use cases of your project